OnChainAgentID: A Provenance-Verified Benchmark for Auditing On-Chain Agent-vs-Human Classifiers

NSF Project Team
University
onchainagentid@anonymous.org

Abstract

Existing benchmarks for detecting automated activity on public blockchains treat label quality as an afterthought: addresses are labeled by the same behavioral heuristics that serve as classifier features, producing apparent AUCs above 0.95 that vanish once label leakage is controlled. We release **OnChainAgentID**, a provenance-verified benchmark of 1,147 Ethereum addresses (533 agents, 614 humans) drawn from 18 operational categories whose labels are anchored to non-behavioral provenance (on-chain registries, relay-attested MEV bots, ENS registration events, Gitcoin donations, PoolTogether deposits, etc.). The benchmark defines a fixed *five-level evaluation protocol*—in-sample fit, random 10-fold CV, temporal holdout, a 70-address strict curated core, and leave-one-platform-out—that exposes three distinct failure modes: label–feature entanglement, temporal drift, and cross-platform non-transfer. We ship a Gebru-style datasheet, frozen train/test splits for every level, a `pip`-installable audit harness (`onchain_audit`) that runs a three-step leakage check on any user-supplied classifier, and baseline results for four tabular models and a graph attention network. On our baselines, pooled AUC falls from 0.98 (in-sample on heuristic labels) to 0.80 (random 10-fold), 0.67 (temporal holdout), 0.77 (strict core), and 0.42 (leave-one-platform-out)—a progression we argue *every* submission to this benchmark should report. The benchmark is released under CC BY 4.0 with a Zenodo DOI; the audit harness is MIT-licensed. Our goal is to give blockchain-ML researchers a shared, leakage-resistant evaluation substrate so that future agent-detection claims can be compared on equal footing and audited for free.

1 Introduction

The gap. Autonomous AI agents now manage billions of dollars on Ethereum. Platforms such as Autonolas, Fetch.ai, AI Arena, and Virtuals Protocol make deployment turnkey, and a growing security literature treats "AI-controlled address" as a first-class category. From the protocol's perspective, however, an agent-driven externally-owned account (EOA) is indistinguishable from a human wallet—no public registry covers agents exhaustively, and no chain-native signal separates automated from manual activity. Reliable on-chain agent detection would enable agent-aware protocol design, security auditing, and regulatory monitoring. But *reliable detection requires a reliable benchmark*, and the current empirical practice in the field has a structural problem.

The problem: label leakage is the norm, not the exception. Published blockchain bot-detection results rely on labels produced by behavioral heuristics (e.g., "more than N transactions per week," "> 1 bit hourly entropy," "inter-transaction coefficient of variation < 1"). The same behavioral quantities then appear, with renaming or an algebraic transform, as classifier features. Resulting models achieve AUC > 0.95 not because they detect agents but because they re-derive the labeling rule. In our own prior work on an agent-labeled corpus of 3,316 addresses, removing three leaked features (`active_hour_entropy`, `burst_frequency`, `tx_interval_std`) dropped gradient-boosted AUC from 0.98 to 0.80. To our knowledge, no existing blockchain classification benchmark ships with a leakage-audit protocol attached.

What we release. **OnChainAgentID** packages four artifacts that together let any researcher run a leakage-resistant agent-detection evaluation end-to-end:

1. **A dataset** of 1,147 Ethereum addresses with 23 behavioral features and provenance-verified binary labels, plus per-address raw transaction histories (4,854 files total across mining pool). Addresses come from 18 operational categories, 9 agent and 9 human, and every label is traceable to a non-behavioral provenance event (registry enrollment, relay attestation, ENS action, or hand curation). Fully documented with a Gebru et al. (2018)-style datasheet.

2. **A fixed five-level evaluation protocol** with frozen JSON splits. Levels range from a sanity-check in-sample fit (Level 1) through random 10-fold CV (Level 2), temporal holdout (Level 3), a 70-address strict curated core with no heuristic labels (Level 4), to leave-one-platform-out over 14 platform folds (Level 5). The five levels probe three orthogonal failure modes (label entanglement, temporal drift, cross-platform non-transfer), and we argue submissions should always report all five.

3. **A `pip`-installable audit harness**, `onchain_audit`, that runs a three-step leakage check (label–feature overlap, label-purity tier comparison, cross-scheme transfer) on any user-supplied classifier and emits a Markdown verdict report. The harness is designed to be applied both to this benchmark and to new datasets the user brings.

4. **Baseline results** for four tabular classifiers (Random Forest, Gradient Boosting, Logistic Regression, LightGBM) and one graph neural network (GAT) at every level, together with a per-level leaderboard template for future submissions.

What a user of the benchmark does.

1. `pip install onchain-audit`; clone the benchmark repo (frozen Zenodo DOI).
2. Load `features_provenance_v4.parquet` and `labels_provenance_v4.json` plus the five split JSONs.
3. Train their classifier on each split and report AUC/F1/precision/recall on all five levels.
4. Run `generate_audit_report(clf, dataset)` to verify no leakage indicator triggers.
5. Submit their numbers to the leaderboard template (Table 3) along with a short methods note and the audit report.

What this paper is not. It is not a new agent-detection *method*. The five-level collapse from 0.98 to 0.42 on our own pipeline is included as a self-audit case study (§5) demonstrating how to apply the toolkit, not as a methodological proposal. The contributions here are the dataset, the evaluation protocol, the harness, and the baselines—artifacts, not algorithms.

2 Related Datasets

The *Elliptic* [11] Bitcoin AML dataset uses coarse `licit/illicit/unknown` labels and ships no fixed cross-platform split; results are known to be sensitive to graph re-splitting. *Ethereum phishing* datasets [5] mix user reports with behavioral rules; Tikhomirov and Ivanitskiy [7] specifically warned that heuristic phishing labels leak address-type features. Victor et al. [9] release MEV-bot behavioral data but no human counterpart, so cross-class distinctions are confounded with class imbalance. Li et al. [4] cover a single bot family. Social-media bot benchmarks [1, 8] report 0.75–0.85 AUC under mixed-source labels, which mask label provenance post-hoc.

Three structural gaps motivated a new benchmark rather than reuse: (i) existing datasets do not publish their labeling rules in machine-checkable form, so label–feature overlap cannot be audited; (ii) published blockchain benchmarks ship at most random-CV splits, so a high number cannot be distinguished from platform fingerprinting; (iii) no public dataset provides a provenance-verified agent-vs-human binary with both classes drawn from ≥ 7 independent sources. Recent surveys [3, 6] catalog agent platforms and GNN-on-blockchain methods but do not release benchmarks; Wang et al. [10] audit AI-agent wallet risks at the prompt-injection level, orthogonal to address classification. OnChainAgentID sits below these as a shared evaluation substrate.

3 Dataset Description

3.1 Instances and fields

Each of the 1,147 instances is a single Ethereum address together with:

- 23 numerical behavioral features, grouped as *temporal* (7), *gas* (6), *interaction* (5), and *approval/security* (5). See §3.4 and Appendix A for the full list.
- A binary `label` (1 = agent, 0 = human).
- Provenance metadata: `provenance_source`, `label_provenance` $\in \{0, 1\}$, and `category`.
- Raw Etherscan transaction history (blockNumber, timeStamp, from, to, value, gas, gasPrice, gasUsed, isError, methodId, ...), up to 10,000 most recent transactions per address, released separately as one parquet per address.

`label_provenance` = 1 indicates an off-chain or registry-level attestation (Flashbots relay, Etherscan label, Arkham label, ENS verification, hand curation); `label_provenance` = 0 indicates a strictly on-chain heuristic such as "called `performUpkeep` on Chainlink Registry ≥ 3 times." Both are traceable to non-behavioral events, in contrast to labels gated on 23-feature-level statistics.

3.2 Source categories

Table 1 lists the 18 operational categories and their provenance basis. The agent class (533 addresses) is dominated by DeFi high-frequency traders and Chainlink keepers; the human class (614 addresses) is dominated by ENS-active and Gitcoin-active accounts. Class ratio is 0.87:1, approximately balanced.

Table 1: The 18 source categories in OnChainAgentID v4 (N=1,147).

Class	Category	N	Provenance basis
Agent	`defi_hf_trader`	199	On-chain rule: > 100 tx/week for > 4 weeks
Agent	`chainlink_keeper`	108	Chainlink Automation Registry interactions
Agent	`compound_v3_liquidator`	97	Compound V3 liquidation function calls
Agent	`gelato_executor`	47	Gelato Network executor contracts
Agent	`keep3r_executor`	46	Keep3r Network job execution
Agent	`pilot_agent`	15	Manual curation (pilot set)
Agent	`curated_mev_bot`	10	Etherscan/Arkham MEV labels + Flashbots relay
Agent	`expanded_mev_bot`	10	Manual MEV curation (expanded pilot)
Agent	`flash_loan_user`	1	Aave/dYdX flash loan function calls
Human	`human_ens_interaction`	167	ENS registration/transfer events
Human	`gitcoin_donor`	146	Gitcoin Grants donation events
Human	`ens_reverse_setter`	144	ENS reverse registrar
Human	`pooltogether_depositor`	89	PoolTogether V4 deposits
Human	`human_exchange_depositor`	28	Known CEX deposit addresses
Human	`expanded_human`	25	Manual curation (expanded pilot)
Human	`curated_human`	7	ENS + Twitter verified
Human	`human_ens_int._governance_voter`	5	ENS + DAO vote participant
Human	`pilot_human`	3	Manual curation (pilot set)

3.3 Collection process

Tier 1—curation (70 addresses). The strict curated core was seeded from Flashbots relay attestations, Etherscan agent tags (jaredfromsubway, Wintermute, Autonolas service operators), and Arkham entity labels for agents; and from ENS verification (e.g., vitalik.eth), Twitter-verified ENS resolvers, and DAO governance voters for humans. No address in the curated core passes through any behavioral threshold.

Tier 2—on-chain heuristic mining (∼1,077 addresses). For each platform (Chainlink, Compound V3, Gelato, Keep3r, Gitcoin, PoolTogether, ENS) we wrote a SQL-like query against the Etherscan V2 API that matches the platform's canonical interaction pattern (e.g., "called `performUpkeep` on `0x...` ≥ 3 times in 30 days"). These heuristics are *provenance*-level: they check for interaction with a specific registry contract, not for statistical properties of the address's behavior.

Feature extraction. We fetch up to 10,000 historical transactions per address from Etherscan and compute the 23 behavioral features via `features/feature_pipeline.py`. NaN values are imputed at the column median and 1st/99th-percentile clipped at train time (not in the stored parquet).

Mining window: 2026-04-07 to 2026-04-09. Transactions span block 18,000,000 to block 22,000,000 (late 2023 through April 2026). No human subjects were involved; no personal data is included except where subjects already self-attested publicly.

3.4 Feature groups

The 23 features split into four groups: *temporal* (7: inter-transaction interval moments, hour-of-day entropy, burst frequency), *gas* (6: gas-price precision statistics, EIP-1559 tip precision, gas–nonce correlation), *interaction* (5: unique contract ratio, Herfindahl concentration, method-ID diversity, contract-to-EOA ratio, n-gram repetition), and *approval/security* (5: unlimited-approve ratio, revoke ratio, unverified-contract approvals, multi-protocol count, flash-loan flag). Full definitions in Appendix A.

3.5 Datasheet summary

A full Gebru et al. [2] datasheet ships at `benchmark/datasheet.md`. Headline points: (i) *motivation*—give blockchain-ML researchers a leakage-auditable substrate; (ii) *composition*—1,147 primary, 70 strict core, 4,854 raw-tx files; both classes from ≥ 7 provenance sources; (iii) *collection*—Etherscan V2 API, no crowdsourcing, no IRB; (iv) *uses*—benchmarking under the five-level protocol and auditing user classifiers; *not for* deanonymization, production fraud detection, or LLM-specific agent detection (see §8); (v) *license*—CC BY 4.0 data, MIT code; (vi) *known biases*—platform imbalance (`defi_hf_trader` = 17%), survivor bias (≥ 10 tx), time-window bias (no pre-2023-11 addresses).

4 Evaluation Protocol: The Five-Level Benchmark

The core methodological contribution of OnChainAgentID is a *fixed* five-level evaluation protocol that every submission should report. Each level probes a distinct failure mode; the gap between levels quantifies the magnitude of the exposed failure. All splits are shipped as frozen JSON in `benchmark/splits/`.

Level 1 (in-sample fit). All 1,147 addresses used for both training and scoring. *Sanity check only*; a score below 0.95 implies a modeling bug.

Level 2 (random stratified 10-fold CV). `level2_random_10fold.json`: 10 stratified folds, seed 42, ∼115 test addresses each. *Report:* mean ± std of AUC, F1, precision, recall. This is an upper bound under within-distribution assumptions.

Level 3 (temporal holdout). `level3_temporal_holdout.json`: split at the median first-seen block (block 12,321,026 ≈ 2021-04-27). Train: 573 addresses (170 agents, 403 humans; 70% human). Test: 574 (363 agents, 211 humans; 63% agent). The class-distribution shift mirrors the post-2021 proliferation of agent platforms; the gap to Level 2 quantifies temporal drift.

Level 4 (strict curated core). `level4_strict_core.json`: 70 hand-curated addresses (35 agents, 35 humans) drawn from `pilot_agent`, `curated_mev_bot`, `expanded_mev_bot`, `curated_human`, `pilot_human`, `expanded_human`. Evaluate via LOO-CV or repeated stratified 5-fold. *Report:* LOO AUC and bootstrap CI95. No label here passes through a behavioral threshold, so Level 4 is the honest ceiling.

Level 5 (leave-one-platform-out, LOPO). `level5_lopo.json`: 14 folds, one per platform category with ≥ 10 addresses. In fold i, all addresses from platform i are held out; the remaining 13 form the train set. *Report:* per-platform accuracy (= per-class recall, since each platform is single-class) plus pooled AUC over the union of held-out predictions.

Structural caveat on Level 5. The single-class-per-platform design conflates domain shift with concept failure, so Level 5 measures *cross-context deployment readiness* rather than pure concept generalization. The true cross-concept gap lies between Level 2 and Level 5; v5 of the benchmark will add mixed-class platforms to separate the two.

4.1 What each level diagnoses

Table 2: Diagnostic mapping between levels and failure modes.

Level pair	Gap measures	Implied failure mode if large
$L_1 \to L_2$	Overfitting	Classifier capacity too large for dataset
$L_2 \to L_3$	Temporal drift	Features/labels non-stationary over time
$L_2 \to L_4$	Heuristic-label inflation	Level-2 labels contain label–feature leakage
$L_2 \to L_5$	Cross-platform non-transfer	Learned features are platform-specific

4.2 Leaderboard template

Table 3 gives the recommended reporting schema. We seed the leaderboard with the baselines from §6; community submissions should append a row.

Table 3: Leaderboard template. Submissions fill one row; audit verdict should cite the harness output.

Model (submitter)	L_1 AUC	L_2 AUC	L_3 AUC	L_4 AUC	L_5 AUC	Audit verdict
Baselines (this paper):						
Random Forest	–	0.803	0.669	0.771	∼0.42	PASS
Gradient Boosting	–	0.810	0.664	0.628	**0.420**	PASS
Logistic Regression	–	0.779	0.616	0.610	–	PASS
LightGBM	–	–	–	0.710	–	PASS
GAT (graph attention)	–	**0.832**	–	**0.883**	–	PASS
Leaky reference (do not cite as SOTA):						
GBM on C1–C4 heuristic labels	–	0.980	–	–	–	FAIL

5 Audit Toolkit

5.1 Installation and use

The `onchain_audit` Python package accompanies the benchmark. A three-line end-to-end audit is:

```
pip install onchain-audit
from onchain_audit import generate_audit_report, load_example_dataset
print(generate_audit_report(my_clf, load_example_dataset()))
```

The output is a Markdown report (writable to disk, pasteable into an appendix) with a **PASS/FAIL** verdict followed by three diagnostic sections: label–feature overlap, label-purity tier comparison, and cross-scheme transfer. On a commodity laptop, the full audit runs in ∼1 minute.

Algorithm 1: `generate_audit_report`: three-step leakage audit.

Input: Features $\mathcal{F}$, labels y, mining rules $\mathcal{R}$, classifier h, purity tiers $\{T_1, \ldots, T_k\}$, schemes (A, B)
Output: Verdict $\in \{\textsc{Pass}, \textsc{Fail}\}$ plus a diagnostic report
Step 1: For each $(r, f) \in \mathcal{R} \times \mathcal{F}$, compute $J(r, f)$ and $|\mathrm{corr}(r, f)|$; flag any pair above threshold.;
Step 2: Train h independently on each tier T_i under 5×3 repeated stratified CV; record a_i. Warn if $\max_i a_i - \min_i a_i \geq 0.15$.;
Step 3: Compute internal CV AUC a_A, a_B; cross-train h on A, test on B, and vice versa, giving $t_{A\to B}, t_{B\to A}$. Warn if $\mathrm{mean}(a_A, a_B) - \mathrm{mean}(t_{A\to B}, t_{B\to A}) \geq 0.15$.;
Verdict: FAIL if any step warns; PASS otherwise.;

5.2 The three-step protocol

Step 1—Label–feature overlap. Given the mining rules used to produce the labels and the classifier features, compute (a) Jaccard similarity between the boolean support of each rule and the median-binarized support of each feature and (b) Pearson and Spearman correlation between each rule and each feature. Risk flag if any pair exceeds $J \geq 0.5$ or $|\rho| \geq 0.6$.

Step 2—Label-purity tier comparison. Evaluate the classifier independently on each label-purity tier. For OnChainAgentID, the default tiers are `all_mined` (both heuristic and curated labels), `curated_only` (`label_provenance` = 1), and `strict_core` (the 70-address Level 4 set). A drop in AUC of ≥ 0.15 between the best and worst tier indicates the classifier's lift depends on label purity.

Step 3—Cross-scheme transfer. Partition the addresses into two schemes whose labels come from independent provenance sources (e.g., on-chain registry labels vs. Flashbots/Arkham attestations). Train on scheme A, evaluate on scheme B, and vice versa. A transfer gap ≥ 0.15 below each scheme's internal CV suggests the classifier learned scheme-specific cues rather than a stable agent-vs-human signal.

Algorithm 1 formalizes the full procedure.

5.3 Self-audit case study (why the harness exists)

We applied the harness to our own prior 3,316-address pipeline, which used a 4-gate labeling oracle (C1–C4) that gated AGENT labels on `hour_entropy > 2.5`, `burst_ratio > 0.1`, and `tx_interval_cv < 1.0`. Three of the top-four classifier features (`active_hour_entropy`, `burst_frequency`, `tx_interval_std`) are algebraically identical or near-identical to the three gate variables. The resulting GBM achieved AUC 0.98, a number that made its way into an earlier draft as the headline.

Running the harness produced the following (abbreviated):

- **Step 1 flag**: three rules have $|\rho| > 0.9$ with features bearing the same name up to renaming.
- **Step 2 flag**: AUC on `all_mined` is 0.98; on `strict_core` (70 addresses), 0.63. Drop of 0.35 (threshold 0.15).
- **Step 3 flag**: training on C1–C4 Autonolas holders and testing on curated MEV bots yields AUC 0.27 (below chance); the reverse transfer yields 0.34.

All three indicators triggered, so the harness verdict was **FAIL**. The fix was to demote C1–C4 to inference-only and build the provenance-verified v4 set that OnChainAgentID now ships. The harness is designed to catch exactly this failure mode for future users *before* the high AUC number becomes a headline. §6 reports the post-fix numbers that Table 3 uses as baselines.

6 Baseline Results

We report baseline numbers from four tabular models (Random Forest, Gradient Boosting, Logistic Regression, LightGBM) and one graph model (Graph Attention Network, GAT) across all five levels. Reproducers are in `experiments/run_*.py`; pinned versions in `requirements.txt`.

6.1 Level 2: Random 10-fold CV on N=1,147

Table 4 reports the core random-CV baseline. GAT is the best overall model (AUC 0.832 ± 0.021) and RF is the best tabular model (AUC 0.803). Standard deviations are small because the per-fold test size is ∼115.

Table 4: Level 2 (random 10-fold) baselines on OnChainAgentID v4 (N=1,147).

Model	Protocol	AUC	Precision	Recall	F1
Random Forest	10-fold	0.803	0.744	0.623	0.678
Random Forest	5×10-CV	0.803 ± 0.027	–	–	–
Gradient Boosting	10-fold	0.810	–	–	0.703
Logistic Regression	5×10-CV	0.779 ± 0.028	0.711	0.658	0.682
GAT (2-layer, 8 heads)	5-fold	**0.832 ± 0.021**	–	–	0.674

Feature-group ablation on RF (all-23 AUC 0.794): interaction group alone 0.751; gas alone 0.713; approval/security alone 0.656; temporal alone 0.623. Removing temporal features alone improves AUC by +0.005—evidence that the heuristic-labeled pipeline used temporal features as a crutch.

6.2 Level 3: Temporal holdout

Table 5 shows a 13–16 AUC-point drop from random CV to temporal holdout across all tabular models. Precision rises (the classifier becomes more conservative) but recall collapses: RF temporal recall is 0.309. Practitioners deploying against post-training-cutoff agents should expect to catch fewer than half of them without retraining.

Table 5: Level 3 (temporal holdout) vs. Level 2, tabular models.

Model	Evaluation	AUC	Precision	Recall	F1
Random Forest	Random 5×10-CV	0.803 ± 0.027	0.755	0.623	0.681
Random Forest	Temporal holdout	0.669	0.812	0.309	0.447
Gradient Boosting	Random 5×10-CV	0.802 ± 0.027	0.725	0.675	0.698
Gradient Boosting	Temporal holdout	0.664	0.767	0.499	0.604
Logistic Regression	Random 5×10-CV	0.779 ± 0.028	0.711	0.658	0.682
Logistic Regression	Temporal holdout	0.616	0.739	0.422	0.537

6.3 Level 4: Strict curated core (N=70)

On the 70-address strict core, RF is again the best tabular model (LOO AUC 0.771); GAT reaches 0.883 with wide fold-to-fold variance. Confidence intervals are wide due to small n; 5×10 repeated CV standard deviations are 0.08–0.12.

Table 6: Level 4 (strict curated core, N=70) baselines.

Model	Protocol	AUC	Prec.	Recall	F1	Accuracy
Random Forest	LOO	0.771	0.742	0.697	0.719	0.719
Random Forest	5×10-CV	0.803 ± 0.102	0.801	0.714	0.736	0.745
Gradient Boosting	LOO	0.628	0.588	0.606	0.597	0.578
Logistic Regression	LOO	0.610	0.667	0.606	0.635	0.641
LightGBM	LOO	0.710	–	–	0.727	0.719
GAT (5-fold)	5-fold	**0.883 ± 0.108**	–	–	–	–

6.4 Level 5: Leave-One-Platform-Out

Pooled over 14 held-out platform folds (1,131 predictions), GBM reaches pooled AUC 0.420 and pooled accuracy 46.4%. Per-platform recalls range from 14% (`defi_hf_trader`) to 80% (`pilot_agent`); human specificities

range from 43% (human_ens_interaction) to 86% (human_exchange_depositor). The pooled AUC below 0.50 confirms that, under the single-class-per-platform LOPO, current features do not transfer.

Table 7: Level 5 (LOPO) per-platform accuracies (GBM baseline, pooled AUC 0.42).

Held-out category	Class	n	Accuracy (%)
pilot_agent	agent	15	**80.0**
chainlink_keeper	agent	108	67.6
gelato_executor	agent	47	61.7
curated_mev_bot	agent	10	60.0
keep3r_executor	agent	46	26.1
expanded_mev_bot	agent	10	20.0
compound_v3_liquidator	agent	97	15.5
defi_hf_trader	agent	199	**14.1**
human_exchange_depositor	human	28	**85.7**
ens_reverse_setter	human	144	76.4
expanded_human	human	25	56.0
gitcoin_donor	human	146	54.8
pooltogether_depositor	human	89	53.9
human_ens_interaction	human	167	43.1
Pooled (all 1,131)	–	1,131	AUC **0.420**

6.5 End-to-end reproducibility

Every number in Tables 4–7 is reproducible in under 30 minutes on a commodity laptop (no GPU needed for tabular models; a single GPU for GAT) via:

```
git clone https://github.com/adelinewen/NSF
cd NSF/paper1_onchain_agent_id
pip install -e .
python experiments/run_provenance_pipeline.py   # Level 2
python experiments/run_temporal_holdout.py       # Level 3
python experiments/run_strict_core.py            # Level 4
python experiments/leave_one_platform_out.py     # Level 5
```

Random seeds are pinned to 42 (Levels 2, 3) and 0 (Level 5). JSON splits are frozen in benchmark/splits/; regenerating them (benchmark/splits/generate_splits.py) is deterministic given the same seed and version of the features parquet.

7 Ethics and Datasheet Summary

All 1,147 addresses are pseudonymous public blockchain identifiers; no personal data is attached. The handful with a real-world identity (e.g., vitalik.eth, Wintermute) are publicly self-attested. No IRB required. *Misuses and mitigations:* (i) *deanonymization*—the benchmark tags only behavioral archetypes, never identities not already public; (ii) *adversarial evasion*—feature importances could inform evasion, an acceptable open-science cost; (iii) *over-reliance on Level 2*—mitigated by the five-level reporting convention, the audit harness, and the leaderboard template. *Positive uses:* agent-aware protocol design, security auditing of agent-held DeFi positions, regulatory monitoring within known distributions, and as a teaching artifact. *License:* CC BY 4.0 (data), MIT (code); Zenodo DOI at camera-ready; erratum policy anchored on dated FINDINGS_YYYY-MM-DD.md.

8 Limitations and Future Work

Chain coverage. Ethereum mainnet only; a Polygon extension (N≈300) from our companion work will merge into v5. *Category imbalance.* defi_hf_trader dominates the agent class (17%) and its mining rule is itself

a behavioral heuristic (> 100 tx/week), so classifiers relying on raw frequency will look artificially strong at Level 2; Step 2 of the audit harness partially flags this. *Single-class platforms at Level 5.* Every platform contributes only one class, conflating domain shift with concept failure; v5 will add mixed-class platforms (e.g., Aave borrowers) to enable a cleaner test. *Label granularity.* The binary agent/human label does not distinguish LLM agents, MEV searchers, liquidation bots, and DeFi management agents; our companion Paper 0 finds LLM agents are behaviorally indistinguishable from DeFi management agents on this feature set (F1 = 0.53), so this benchmark is *not* recommended for LLM-specific detection. *Feature set.* The 23 features are a specific choice; graph-topological and cross-protocol features are absent, but the benchmark supports user-supplied features via the raw parquets.

Future directions: (1) v5 mixed-class platforms; (2) cross-chain extension (Polygon, Base, Arbitrum) as a Level 6; (3) adversarial stress tests with synthetic evasive-agent traces; (4) an LLM-agent subset drawing from ElizaOS / Virtuals once density permits; (5) rolling temporal splits to establish a retraining cadence.

9 Conclusion

OnChainAgentID packages a 1,147-address provenance-verified Ethereum dataset, a five-level evaluation protocol with frozen splits, a `pip`-installable leakage-audit harness, and baseline results for four tabular models and a graph attention network. The benchmark's design responds to a structural gap in blockchain classification research: existing datasets do not publish machine-checkable mining rules, do not ship cross-platform splits, and do not bundle a leakage-auditing tool. We hope the combination of frozen splits, a datasheet, a leaderboard template, and a one-command audit harness lowers the activation energy for honest agent-detection evaluation. The five-level collapse we observe on our own baselines ($0.98 \rightarrow 0.80 \rightarrow 0.67 \rightarrow 0.77 \rightarrow 0.42$) is offered not as a method contribution but as a self-audit case study: an existence proof that without all five levels, high agent-detection AUCs cannot be distinguished from label or platform fingerprints. All artifacts are released under CC BY 4.0 (data) and MIT (code), with a Zenodo DOI issued at camera-ready.

Reproducibility Checklist

- **Data URL (primary):** `https://github.com/adelinewen/NSF/tree/main/paper1_onchain_agent_id`
- **Zenodo DOI:** to be assigned on camera-ready; placeholder in `CITATION.cff`.
- **Audit harness:** `pip install onchain-audit` (MIT, source at `onchain_audit/`).
- **Splits:** `benchmark/splits/level{2,3,4,5}_*.json`, frozen, regenerable via `generate_splits.py`.
- **Baselines:** `experiments/run_*.py`; pinned versions in `requirements.txt`.
- **License:** CC BY 4.0 (data, `benchmark/LICENSE`); MIT (code, root `LICENSE`).
- **Datasheet:** `benchmark/datasheet.md` (Gebru et al. 2018 schema).
- **Erratum policy:** dated `FINDINGS_YYYY-MM-DD.md` at repo root.

References

[1] Stefano Cresci, Roberto Di Pietro, Marinella Petrocchi, Angelo Spognardi, and Maurizio Tesconi. The paradigm-shift of social spambots: Evidence, theories, and tools for the arms race. In *Proc. of the Web Conference Companion (WWW)*, 2017.

[2] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *arXiv preprint arXiv:1803.09010*, 2018.

[3] Yu He et al. A survey on AI agents on blockchain. *arXiv:2601.04583*, 2026.

[4] Yichen Li, Stefanos Chaliasos, and Benjamin Livshits. Telegram trading bots: An empirical study. In *Proc. Financial Cryptography (FC)*, 2024.

[5] V. Patel et al. GNN-based ethereum phishing detection. In *Proc. BigData*, 2021.

[6] Kiarash Shamsi et al. A survey of GNNs for blockchain analytics. *arXiv preprint*, 2023.

[7] Sergei Tikhomirov and Valery Ivanitskiy. Detecting phishing addresses on Ethereum via transaction graph. In *Proc. of the Web Conference (WWW) Companion*, 2021.

[8] Onur Varol, Emilio Ferrara, Clayton A. Davis, Filippo Menczer, and Alessandro Flammini. Online human-bot interactions: Detection, estimation, and characterization. In *Proc. ICWSM*, 2017.

[9] Friedhelm Victor, Andrea Weintraud, and Merve Haliloglu. Behavioral fingerprinting of blockchain bots. *arXiv preprint*, 2023.

[10] X. Wang et al. Risks of AI agents with cryptocurrency access. *arXiv preprint*, 2024.

[11] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD Anti-Money Laundering Workshop*, 2019.

A Full Feature Definitions

Let $T = \{t_1, \ldots, t_n\}$ be transaction timestamps, $G = \{g_1, \ldots, g_n\}$ gas prices, and $C = \{c_1, \ldots, c_n\}$ target contracts.

Temporal (7). (1) `tx_interval_mean`, mean of consecutive inter-transaction intervals. (2) `tx_interval_std`, their std. (3) `tx_interval_skewness`, their skew. (4) `active_hour_entropy`, Shannon entropy of tx bucketed into 24 UTC hours. (5) `night_activity_ratio`, fraction of tx in UTC 0:00–6:00. (6) `weekend_ratio`, fraction on Sat/Sun. (7) `burst_frequency`, fraction of consecutive pairs with interval < 10 s.

Gas (6). (8) `gas_price_round_number_ratio`. (9) `gas_price_trailing_zeros_mean`. (10) `gas_limit_precision` (gas_limit relative to gas_used). (11) `gas_price_cv`. (12) `eip1559_priority_fee_precision`. (13) `gas_price_nonce_correlat`

Interaction (5). (14) `unique_contracts_ratio`. (15) `top_contract_concentration` (HHI). (16) `method_id_diversity`. (17) `contract_to_eoa_ratio`. (18) `sequential_pattern_score` (n-gram repetition).

Approval/security (5). (19) `unlimited_approve_ratio`, fraction of `approve()` at MaxUint256. (20) `approve_revoke_rat` (21) `unverified_contract_approve_ratio`. (22) `multi_protocol_interaction_count`. (23) `flash_loan_usage`.

B Split JSON Schemas

All five split JSONs share a top-level schema:

```
{
  "level": 2,
  "name": "random_stratified_10fold",
  "seed": 42,
  "version": "v4",
  "addresses_file": "benchmark/all_addresses.json",
  "folds": [
    {
      "fold_id": 0,
```

```
      "train_addresses": ["0x...", ...],
      "test_addresses":  ["0x...", ...]
    },
    ...
  ]
}
```

Level 4 uses a flat `addresses` + `labels` pair instead of folds, because LOO is the recommended evaluation.

C Known Errata

As of 2026-04-15: the only erratum is the C1–C4 leakage discovery documented in `FINDINGS_2026-04-08.md`, which motivated the switch from C1–C4 labels (3,316 addresses) to the provenance-verified v4 set (1,147 addresses) that this benchmark ships. The prior heuristic-labeled corpus is retained for inference-only use and never cited as a ground-truth source. Future errata will be appended here and cross-referenced from `CITATION.cff`.